

Quantum State Visualization in Qiskit

Mithilesh Kumar^{1*}, Vivek Maadoori^{2†}

Krea University, Sri City

★ mithilesh.kumar@krea.edu.in, † vivek_maadoori.sias19@krea.ac.in

Abstract

This qiskit-based library provides implementations of various quantum state visualization methods.

Copyright attribution to authors.

This work is a submission to SciPost Physics Codebases.

License information to appear upon publication.

Publication information to appear upon publication.

Received Date

Accepted Date

Published Date

1

Contents

3	1 Introduction	2
4	2 Related Software	3
5	3 Library Architecture	3
6	3.1 Module Structure	3
7	3.2 Design Principles	3
8	3.3 Dependency Graph	4
9	4 Visualization Modalities	4
10	4.1 Bloch Sphere	4
11	4.2 State City	5
12	4.3 Cost Landscape	5
13	4.4 Circuit Diagrams	6
14	4.5 Dynamic Flow	6
15	5 Interactive Dashboard and Hardware Integration	6
16	5.1 Web Dashboard	6
17	5.2 IBM Quantum Hardware Integration	6
18	6 Example Applications	7
19	6.1 MaxCut Landscape for a Triangle Graph	7
20	6.2 H ₂ VQE Energy Surface	7
21	6.3 Bell State Density Matrix	7
22	6.4 Rabi Oscillations	7
23	7 Installation and Testing	8
24	7.1 Installation	8
25	7.2 Testing	8

26	7.3 Documentation	8
27	8 Conclusions and Outlook	8
28	9 A Section	9
29	10 Another Section	9
30	10.1 A first subsection	9
31	10.2 A note about figures	9
32	11 Conclusion	9
33	A First appendix	10
34	B About references	10
35		
36		

1 Introduction

Quantum computing has matured rapidly from a theoretical curiosity into a technology with near-term practical applications, driven by variational hybrid algorithms such as the Quantum Approximate Optimization Algorithm (QAOA) [?] and the Variational Quantum Eigensolver (VQE) [?]. Alongside this technical maturation, the need to develop strong geometric and physical intuition about quantum states has become increasingly important for practitioners working on algorithm design, debugging variational circuits, and communicating results to broader audiences.

Despite the existence of mature quantum simulation frameworks—most notably Qiskit [?], Cirq [?], PennyLane [?], and QuTiP [?—these tools are primarily optimized for computation rather than visualization. The built-in plotting utilities they provide are often limited in scope, require non-trivial boilerplate to invoke, and lack interactive or multi-modal presentation capabilities. Users wishing to visualize density matrices, cost landscapes, or time evolution frequently must either write custom Matplotlib routines or string together disparate utilities across multiple packages.

`quantumviz` fills this gap by providing a unified, composable visualization toolkit that covers the most common representational needs arising in quantum algorithm development. The library is structured around a small set of high-level functions and an optional command-line interface (CLI), so that a researcher can obtain a publication-quality figure with a single function call or a one-line shell command. An optional web dashboard further lowers the barrier for interactive exploration, requiring no Python programming knowledge beyond launching a server process.

The remainder of this paper is organized as follows. Section 2 situates `quantumviz` among related visualization software. Section 3 describes the library architecture and module structure. Section 4 documents each visualization modality in detail. Section 5 covers the interactive dashboard and hardware integration. Section 6 presents representative example applications. Section 7 describes installation and testing. Section 8 concludes with a summary and outlook.

2 Related Software

A number of existing tools offer partial overlap with the capabilities of quantumviz. Table 1 provides a structured comparison.

Table 1: Comparison of visualization capabilities across quantum software packages. A checkmark indicates native support; “partial” indicates limited or workflow-coupled support.

Package	Bloch sphere	Density matrix	Cost landscape	Circuit	Dashboard
quantumviz (this work)	✓	✓ (3D)	✓	✓	✓
Qiskit	✓	partial	—	✓	—
QuTiP	✓	✓	—	—	—
PennyLane	✓	—	partial	✓	—
Cirq	—	—	—	✓	—

Qiskit provides a Bloch sphere renderer (via `statevector_to_bloch`) and an excellent circuit drawer, but offers no cost landscape or density matrix visualization. QuTiP’s Bloch class is well-regarded and its Wigner and Hinton functions support density-matrix inspection, but it does not target gate-based circuit visualization or variational landscapes. PennyLane includes cost-surface plotting utilities for two-parameter circuits, but these are tightly coupled to its gradient-based optimization workflow. None of the above packages provides an out-of-the-box web dashboard for interactive state exploration.

quantumviz is therefore distinguished by its breadth of coverage across all major visualization categories, its unified API, and its optional interactive frontend—all accessible through a single `pip install`.

3 Library Architecture

3.1 Module Structure

quantumviz is organized as a single top-level package (`src/quantumviz/`) with the following module layout:

`src/quantumviz/` |— `__init__.py` – public API surface |— `bloch.py` – Bloch sphere renderer |— `state_city.py` – density matrix 3-D bar chart |— `cost_landscape.py` – QAOA / VQE landscape plotter |— `circuit.py` – circuit diagram rendering |— `dynamic_flow.py` – time evolution Rabi oscillations |— `dashboard/` – FastAPI web server (optional) |— `app.py` |— `routes.py` |— `hardware.py` – IBM Quantum integration (optional)

The public API is surfaced through `__init__.py` and consists of five functions—`plot_bloch_sphere`, `plot_state_city`, `plot_cost_landscape`, `plot_circuit`, and `plot_dynamic_flow`—as well as two utility helpers: `state_to_density` (converts a complex state vector to a density matrix) and `density_to_bloch` (extracts Bloch vector components from a density matrix).

3.2 Design Principles

The library is designed around four guiding principles:

- **Minimal dependencies for the core package.** The mandatory requirements are restricted to `numpy` ≥ 1.20 and `matplotlib` ≥ 3.5 . All optional capabilities—the dashboard (FastAPI, Uvicorn, Pydantic) and hardware support (Qiskit, Qiskit-IBM-Runtime)—are available via extras and are never imported unless explicitly requested.
- **Input format flexibility.** Each visualization function accepts the most natural input format for its domain. For example, the Bloch sphere renderer parses ket notation, spherical angle specifications, and raw Cartesian Bloch vectors from a plain-text file without requiring the user to construct any library objects.
- **CLI parity.** Every Python API function has an equivalent CLI subcommand, enabling integration into shell scripts and analysis pipelines without writing any Python code.
- **Testability.** The test suite (`tests/`) provides unit tests for each module and a coverage-enabled integration test that exercises the full API surface with small synthetic inputs.

3.3 Dependency Graph

Figure 1 (schematic) illustrates the dependency structure. The *core ring* (`bloch`, `state_city`, `cost_landscape`, `circuit`, `dynamic_flow`) depends only on `numpy` and `matplotlib`. The *dashboard ring* additionally depends on FastAPI and Pydantic. The *hardware ring* depends on Qiskit and is always optional. This structure ensures that users who install the base package (`pip install quantumviz`) obtain a lightweight footprint with no framework dependencies.

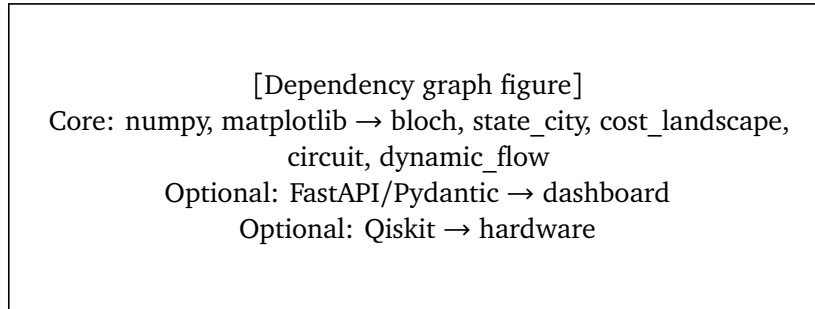


Figure 1: Dependency structure of `quantumviz`. Solid boxes denote mandatory dependencies; dashed borders denote optional extras.

4 Visualization Modalities

4.1 Bloch Sphere

The Bloch sphere is the standard geometric representation of a pure single-qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, parameterized by the polar angle θ and azimuthal angle φ as

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}|1\rangle. \quad (1)$$

Mixed states are represented as points strictly inside the unit sphere, with the Bloch vector given by $\mathbf{r} = (\langle X \rangle, \langle Y \rangle, \langle Z \rangle) = \text{Tr}(\boldsymbol{\sigma}\rho)$ for density matrix ρ . The `bloch.py` module renders a three-dimensional spherical surface using Matplotlib's `Axes3D`, overlays the state vector as an arrow, and optionally annotates the north ($|0\rangle$) and south ($|1\rangle$) poles.

The input file parser supports three notations:

119 • Ket notation, e.g. $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$
 120 • Bloch angles, e.g. $\theta=60$ deg, $\phi=45$
 121 • Cartesian Bloch vector, e.g. (x,y,z)
 122 Example Python usage: `from quantumviz import plot_bloch_sphere`
 123 `plot_bloch_sphere('state.txt', 'bloch.png')`
 124 Equivalent CLI invocation: `quantumviz bloch-sphere state.txt -o bloch.png`

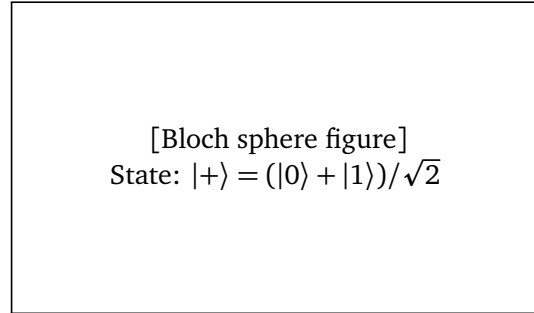


Figure 2: Bloch sphere visualization of the state $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, generated by quantumviz from a single-line input file.

125 4.2 State City

126 The State City plot visualizes the density matrix ρ of a multi-qubit system as a three-dimensional
 127 bar chart in which each bar represents the absolute value of one matrix element $|\rho_{ij}|$. The
 128 height of bar (i, j) encodes the magnitude of the coherence or population, while a color map
 129 encodes the complex argument $\arg(\rho_{ij})$.

130 The `state_city` module accepts either a raw state vector (from which it computes $\rho = |\psi\rangle\langle\psi|$)
 131 or a JSON file containing an explicit `stages` array, allowing visualization of the density matrix
 132 at multiple stages of a quantum circuit:

133 "qubits": 2, "stages": [{"name": "After H gate", "state_vector": [0.707, 0, 0, 0.707] }]

134 For an n -qubit system the density matrix ρ is a $2^n \times 2^n$ matrix. The 3D rendering scales well
 135 for up to $n = 4$ qubits (256 bars) before the figure becomes visually crowded; for larger systems
 136 the user is advised to inspect reduced density matrices or to use the Hinton representation
 137 available as a future extension.

138 4.3 Cost Landscape

139 Variational quantum algorithms optimize a cost function $\mathcal{C}(\theta)$ over a set of circuit parameters θ .
 140 Understanding the geometry of this landscape—in particular the presence of barren plateaus,
 141 local minima, and saddle points—is essential for diagnosing optimizer behavior. quantumviz
 142 supports two problem types for two-parameter landscapes:

- 143 • **QAOA for MaxCut.** Given a graph $G = (V, E)$ specified as an edge list, the module
 144 computes the expectation value $\langle \mathcal{C} \rangle(\gamma, \beta)$ of the MaxCut cost operator $\mathcal{C} = \sum_{(u,v) \in E} \frac{1 - Z_u Z_v}{2}$
 145 over a single-layer QAOA ansatz as a function of parameters $\gamma \in [0, \pi]$ and $\beta \in [0, \pi/2]$.
- 146 • **VQE energy surface.** Given a molecular Hamiltonian expressed as a linear combination
 147 of Pauli terms $H = \sum_k c_k P_k$ (with P_k a tensor product of single-qubit Pauli operators),
 148 the module computes $\langle H \rangle(\theta_1, \theta_2)$ over a two-parameter parameterized state and renders
 149 the result as a filled contour plot.

Input for the QAOA variant is a JSON file of the form: "edges": [[0, 1], [1, 2], [0, 2]]
 Input for the VQE variant follows the Pauli term format: "terms": [{"coeff": -1.0, "paulis":
 [], "coeff": 0.5, "paulis": ["Z"]}]
 Both produce a Matplotlib figure with labeled axes and a color bar indicating the energy
 scale. Example files for the triangle MaxCut graph, the H₂ molecule, and the LiH molecule
 Hamiltonian are included in the examples/ directory.

4.4 Circuit Diagrams

quantumviz renders quantum circuits specified as JSON into PNG images. The JSON schema
 follows a simple gate-list model: each circuit object contains a qubits count and an ordered list
 of stages, where each stage carries a gate label and a list of target qubit indices. The renderer
 uses Matplotlib's text and patch primitives to draw wires, gate boxes, controlled-NOT symbols,
 and measurement boxes.

The circuit module is intentionally lightweight—it does not perform simulation—and is
 designed to be used alongside Qiskit or another simulator rather than to replace it. Its primary
 purpose is to produce readable diagrams for figures and the web dashboard.

4.5 Dynamic Flow

The dynamic flow module visualizes temporal evolution of a quantum state. It accepts the
 same multi-stage JSON schema as State City and renders a sequence of Bloch sphere snapshots
 (or an animated GIF if the animate flag is set) showing how the Bloch vector moves under the
 specified sequence of operations.

A dedicated Rabi oscillation renderer is also provided. Given a coupling strength Ω and a
 detuning Δ , it computes the occupation probability

$$P_0(t) = \cos^2\left(\frac{\Omega_{\text{eff}} t}{2}\right), \quad \Omega_{\text{eff}} = \sqrt{\Omega^2 + \Delta^2}, \quad (2)$$

and plots this as a function of time alongside the trajectory on the Bloch sphere.

5 Interactive Dashboard and Hardware Integration

5.1 Web Dashboard

The optional dashboard extra provides a browser-based interface built on FastAPI and served
 by Uvicorn. It can be launched with: quantumviz serve or equivalently via the Python API:
 from quantumviz.dashboard import start_server start_server(host='0.0.0.0', port=8000)

The dashboard exposes a REST API at /api/v1/ with endpoints for each visualization type.
 Users can upload state vectors or circuit definitions via a web form and receive rendered figures
 in real time. The interface is implemented using vanilla HTML, CSS, and JavaScript with no
 front-end build step required. This makes it easy to embed the dashboard in a Jupyter-adjacent
 workflow or to host it as a classroom tool.

5.2 IBM Quantum Hardware Integration

When installed with the all extra (pip install quantumviz[all]), the hardware module
 provides a thin wrapper around the Qiskit Runtime that allows users to submit a circuit JSON to
 IBM Quantum hardware and retrieve the resulting shot-level counts for subsequent visualization.
 Authentication is handled via the standard IBM Quantum token mechanism. This integration is

188 deliberately minimal: `quantumviz` performs no hardware-specific optimization and makes no
 189 assumptions about the target device topology.

190 6 Example Applications

191 6.1 MaxCut Landscape for a Triangle Graph

192 As a first example, we reproduce the canonical single-layer QAOA cost landscape for the triangle
 193 graph K_3 with three qubits. Using the input file `examples/qaoa_maxcut_triangle.json`,
 194 which encodes the edge set $\{(0, 1), (1, 2), (0, 2)\}$, we generate the landscape with: `quantumviz`
 195 `cost-landscape qaoa examples/qaoa_maxcut_triangle.json -o triangle.png`

196 The resulting figure shows the characteristic sinusoidal structure of the single-layer QAOA
 197 landscape, with the maximum expectation value $\langle C \rangle \approx 2.5$ achieved at the known optimal
 198 angles $(\gamma^*, \beta^*) \approx (\pi/4, \pi/8)$. This serves as a basic sanity check for any QAOA implementation.

199 6.2 H_2 VQE Energy Surface

200 The file `examples/vqe_h2.json` encodes the hydrogen molecule Hamiltonian at a fixed bond
 201 length in the STO-3G basis as a sum of five Pauli terms. The two-parameter energy surface
 202 generated by `quantumviz` reveals a smooth landscape with a single global minimum near the
 203 Hartree–Fock energy, confirming that a gradient-based optimizer would converge reliably from
 204 a broad initialization region.

205 6.3 Bell State Density Matrix

206 The Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ has a density matrix with four non-zero elements:
 207 $\rho_{00} = \rho_{11} = \rho_{03} = \rho_{30} = 0.5$. The State City plot of this state provides a clear visual illustration
 208 of maximal quantum entanglement, with two towers of equal height on the diagonal and two
 209 off-diagonal coherence towers of the same magnitude. The plot is generated with: `from`
 210 `quantumviz import plot_state_city import numpy as np`
 211 `bell = [1/np.sqrt(2), 0, 0, 1/np.sqrt(2)] plot_state_city(bell, r"Bell state  $|\Phi^+\rangle$ ", "bell.png")`

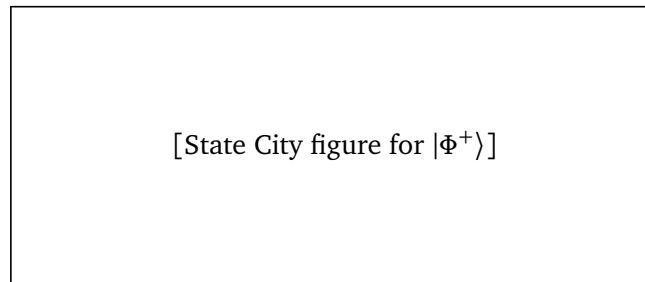


Figure 3: State City visualization of the Bell state $|\Phi^+\rangle$. Bar heights encode $|\rho_{ij}|$ and bar colors encode $\arg(\rho_{ij})$.

212 6.4 Rabi Oscillations

213 The dynamic flow module's Rabi renderer is demonstrated by setting a coupling strength
 214 $\Omega = 2\pi \times 10$ MHz and zero detuning $\Delta = 0$, yielding on-resonance oscillations between $|0\rangle$
 215 and $|1\rangle$ with period $T = 1/\Omega$. The resulting time-series plot provides an immediate visual
 216 verification of the correct Rabi frequency in a simulated or experimental dataset.

7 Installation and Testing

7.1 Installation

quantumviz requires Python ≥ 3.9 . The core package is installed via pip: `pip install quantumviz`

The optional dashboard and hardware extras are available as: `pip install quantumviz[dashboard]`
FastAPI, Uvicorn, Pydantic `pip install quantumviz[all]` all optional dependencies

For development, the repository should be cloned and installed in editable mode: `git clone https://github.com/QuantumVisualization/quantumviz.git cd quantumviz pip install -e ".[dev,all]"`

The `pyproject.toml` build file uses the standard PEP 517/518 build system and is compatible with both pip and build.

7.2 Testing

The test suite is located in `tests/` and is invoked with: `pytest pytest --cov=quantumviz --cov-report=html` with coverage report

Tests are organized by module and cover:

- Input parsing for all supported file formats.
- Correctness of the density-matrix conversion utility against analytical results.
- Correct extrema of the QAOA landscape for the triangle graph.
- Smoke-test rendering (verifying that no exceptions are raised and that output files are produced with non-zero size) for all six visualization types.

Code quality is maintained through Ruff (linting) and Mypy (type checking), both configured in `pyproject.toml` and enforced by a GitHub Actions CI workflow (`.github/workflows/`).

7.3 Documentation

User-facing documentation is hosted in the `docs/` directory in Markdown format and covers installation, all CLI subcommands, Python API reference, and the JSON input schemas. Extended usage examples are provided in the `examples/` directory:

- `examples/qaoa_maxcut_2qubit.json` — two-qubit MaxCut
- `examples/qaoa_maxcut_triangle.json` — triangle MaxCut
- `examples/vqe_h2.json` — hydrogen molecule
- `examples/vqe_lih.json` — lithium hydride molecule

8 Conclusions and Outlook

We have presented quantumviz, a Python library providing a unified interface for the visualization of quantum algorithm states, density matrices, cost landscapes, circuit diagrams, and time evolution. The library is designed to be lightweight, composable, and accessible via both a Python API and a CLI, with an optional web dashboard for interactive use. It addresses a genuine gap in the existing quantum software ecosystem by combining multiple visualization modalities under a single, consistently designed interface.

Several extensions are planned for future releases:

- Multi-qubit Bloch sphere trajectories, showing the reduced Bloch vector of individual qubits evolving within a larger entangled system.
- Wigner function rendering for continuous-variable and bosonic system states.
- Animated GIF and MP4 export for all time-dependent visualizations.
- A Hinton diagram alternative to State City for high-dimensional density matrices.
- Native integration with PennyLane datasets and Cirq circuits.

The source code is available at <https://github.com/QuantumVisualization/quantumviz> under the MIT license. Contributions, bug reports, and feature requests are welcome via the repository issue tracker.

9 A Section

Use sections to structure your article's presentation.

Equations should be centered; multi-line equations should be aligned.

$$H = \sum_{j=1}^N \left[J(S_j^x S_{j+1}^x + S_j^y S_{j+1}^y + \Delta S_j^z S_{j+1}^z) - h S_j^z \right]. \quad (3)$$

In the list of references, cited papers [?] should include authors, title, journal reference (journal name, volume number (in bold), start page) and most importantly a DOI link. For a preprint [?], please include authors, title (please ensure proper capitalization) and arXiv link. If you use BiBTeX with our style file, the right format will be automatically implemented.

All equations and references should be hyperlinked to ensure ease of navigation. This also holds for [sub]sections: readers should be able to easily jump to Section 10.

10 Another Section

There is no strict length limitation, but the authors are strongly encouraged to keep contents to the strict minimum necessary for peers to reproduce the research described in the paper.

10.1 A first subsection

You are free to use dividers as you see fit.

10.2 A note about figures

Figures should only occupy the strictly necessary space, in any case individually fitting on a single page. Each figure item should be appropriately labeled and accompanied by a descriptive caption. **SciPost does not accept creative or promotional figures or artist's impressions**; on the other hand, technical drawings and scientifically accurate representations are encouraged.

11 Conclusion

You must include a conclusion.

Acknowledgements

Acknowledgements should follow immediately after the conclusion.

Author contributions This is optional. If desired, contributions should be succinctly described in a single short paragraph, using author initials.

Funding information Authors are required to provide funding information, including relevant agencies and grant numbers with linked author's initials. Correctly-provided data will be linked to funders listed in the [Fundref registry](#).

A First appendix

Add material which is better left outside the main text in a series of Appendices labeled by capital letters.

B About references

Your references should start with the comma-separated author list (initials + last name), the publication title in italics, the journal reference with volume in bold, start page number, publication year in parenthesis, completed by the DOI link (linking must be implemented before publication). If using BiBTeX, please use the style files provided on https://scipost.org/submissions/author_guidelines. If you are using our LaTeX template, simply add

```
\bibliography{your_bibtex_file}
```

at the end of your document. If you are not using our LaTeX template, please still use our bibstyle as

```
\bibliographystyle{SciPost_bibstyle}
```

in order to simplify the production of your paper.